子序列与排列：五道题的区别和联系

最长公共子序列 · 最长上升子序列 · 最大上升子序列和 · 排列最少移动 · 最长锯齿子序列

核心方法：先明确哪些元素可以保留，再定义状态记录“已经处理到哪里”和“最后一步满足什么条件”。这些题都允许跳过元素，但目标量和转移条件各不相同。

一眼区分五道题

题目	状态记录什么	关键条件	答案
最长公共子序列	两串的前缀	字符相等	最大长度
最长上升子序列	以 a[i] 结尾	前值 < 当前值	最大长度
最大上升子序列和	以 a[i] 结尾	前值 < 当前值	最大元素和
排列最少移动	以数值 x 结尾的连续值段	pos[x-1] < pos[x]	n - 最长保留段
最长锯齿子序列	以 a[i] 结尾 + 末步方向	上升、下降交替	最大长度

共同前提

子序列不要求在原序列中连续，但所选元素的相对顺序不能改变。严格上升必须使用 <，相等不能算上升；锯齿序列要求相邻差值正负交替。排列则指 1 到 n 各出现一次，因而可以用 pos[x] 唯一记录数值 x 的位置。

阅读顺序：先看状态的含义，再看为什么可以转移，最后对照源代码。所有代码均按 C++17 编写。

01 最长公共子序列 LCS

问题：给定两个字符串，求它们的最长公共子序列长度。样例 ABCB 与 BDCA 的答案是 2，例如公共子序列 BC。

状态与转移

令 dp[i][j] 表示第一个字符串前 i 个字符、第二个字符串前 j 个字符的 LCS 长度。若 a[i-1] == b[j-1]，这对相同字符可以共同接到前缀答案后面：dp[i][j] = dp[i-1][j-1] + 1。否则至少舍弃其中一个末尾字符：dp[i][j] = max(dp[i-1][j], dp[i][j-1])。任一前缀为空时答案为 0。

注意两个下标各自对应一条字符串：比较 a[i-1] 与 b[j-1]，不能写成 b[i-1]。

a \ b	空	B	D	C	A
空	0	0	0	0	0
A	0	0	0	0	1
B	0	1	1	1	1
C	0	1	1	2	2
B	0	1	1	2	2

样例 DP 表：右下角的 2 即最终答案。

完整源代码

```
#include <algorithm>
#include <iostream>
#include <string>
using namespace std;

int dp[101][101];

int main() {
    string a, b;
    cin >> a >> b;
    int n = a.size(), m = b.size();
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    }
    cout << dp[n][m] << '\n';
}
```

复杂度：时间 O(|a|×|b|)，空间 O(|a|×|b|)。字符串长度不超过 100，二维数组足够。

02 最长上升子序列 LIS

问题：在一个序列中选出尽量多的数，使其按原顺序严格递增。样例 1,7,3,5,9,4,8 的答案是 4，例如 1,3,5,8。

标准动态规划：O(n²)

dp[i] 表示以 a[i] 结尾的 LIS 长度。先令 dp[i] = 1；枚举 j < i，若 a[j] < a[i]，则 a[i] 可以接到以 a[j] 结尾的子序列后面：dp[i] = max(dp[i], dp[j] + 1)。答案是所有 dp[i] 的最大值，而不是 dp[n-1]。

若需输出具体子序列，在每次得到更优转移时记 pre[i] = j。从最佳结尾沿 pre 倒推，再反转。下面的代码已保存 pre；原题只要求长度，因此还原代码保留为注释。

位置	1	2	3	4	5	6	7
a[i]	1	7	3	5	9	4	8
dp[i]	1	2	2	3	4	3	4

样例中 dp[5]=4，而 dp[7]=4；答案需要取所有结尾的最大值。

完整源代码：DP 与还原入口

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n), dp(n, 1), pre(n, -1);
    for (int &x : a) cin >> x;

    int endPos = 0;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                pre[i] = j;
            }
        }
        if (dp[i] > dp[endPos]) endPos = i;
    }
    cout << dp[endPos] << '\n';

    // If the problem asks for a sequence, uncomment:
    // vector<int> seq;
    // for (int i = endPos; i != -1; i = pre[i]) seq.push_back(a[i]);
    // reverse(seq.begin(), seq.end());
    // for (int x : seq) cout << x << ' ';
}
```

复杂度：时间 O(n²)，空间 O(n)。n ≤ 1000 时足够。

02 LIS 的 O(n log n) 做法

它是贪心维护“各长度的最小结尾值”，并用二分查找更新位置；与上一页按位置枚举前驱的标准 DP 不同。

为什么只保留最小结尾

设 tail[k] 是长度为 k+1 的上升子序列可能取得的最小结尾。若两个子序列长度相同，结尾较小的那条至少同样容易接上未来的数，所以只需保留更小的结尾。读入 x 时，用 lower_bound 找第一个 ≥ x 的位置：存在就替换，不存在就追加。严格上升不能把相等的数追加。

样例的 tail 变化： [1] → [1,7] → [1,3] → [1,3,5] → [1,3,5,9] → [1,3,4,9] → [1,3,4,8]。数组长度为 4。

tail 的值不一定构成原序列的某一条真实子序列。例如 2,3,1 处理后 tail = [1,3]，但 1 在 3 后面；此处只能直接读取长度。若要还原，需要额外记录各长度的结尾下标和每个元素的前驱下标。

完整源代码：只输出长度

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> tail;
    for (int i = 0; i < n; ++i) {
        int x;
        cin >> x;
        auto it = lower_bound(tail.begin(), tail.end(), x);
        if (it == tail.end()) tail.push_back(x);
        else *it = x;
    }
    cout << tail.size() << '\n';
}
```

复杂度：时间 O(n log n)，空间 O(n)。

03 最大上升子序列和

问题：仍要求严格上升，但最大化选中元素的和，而非元素个数。样例 1,7,3,5,9,4,8 的答案是 18，对应 1,3,5,9。

与 LIS 的联系：只换目标量

dp[i] 表示以 a[i] 结尾的上升子序列的最大和。初值是 a[i]，表示只选当前数。若 j < i 且 a[j] < a[i]，则 dp[i] = max(dp[i], dp[j] + a[i])。最终对所有结尾取最大值。与 LIS 的结构完全相同，只是“+1”变为“+a[i]”。

最大和不必来自最长子序列：在 100,1,2,3 中，最长上升子序列是 1,2,3，和为 6；最大上升子序列和是单独选择 100。

位置	1	2	3	4	5	6	7
a[i]	1	7	3	5	9	4	8
dp[i]	1	8	4	9	18	8	17

样例中以数字 9 结尾的最优和为 18。

完整源代码

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n), dp(n);
    for (int &x : a) cin >> x;

    int ans = 0;
    for (int i = 0; i < n; ++i) {
        dp[i] = a[i];
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i])
                dp[i] = max(dp[i], dp[j] + a[i]);
        }
        ans = max(ans, dp[i]);
    }
    cout << ans << '\n';
}
```

复杂度：时间 O(n²)，空间 O(n)。本题数值非负且 n ≤ 1000，int 足够容纳答案。

04 排列升序：最少移到两端几次

问题：每次选一个数移到开头或结尾，使排列最终为 1,2,…,n，求最少操作次数。样例 3,5,4,6,1,7,2 的答案是 4。

先把最少移动转为最多保留

没有被移动的数，其相对顺序保持不变，而且最终处于中间的一段。因此，这些数必须是数值连续的 L,L+1,…,R，并且在原排列中的位置递增。若能保留最长的 len 个数，就只需移动其余 n-len 个数：把更小的数按从大到小的顺序移到开头，把更大的数按从小到大的顺序移到结尾。

按数值做 DP，而非按原位置做 DP

因为输入是 1 到 n 的排列，每个数恰好出现一次。读入数字 x 时记录 pos[x] = 它的原位置。令 dp[x] 为以数值 x 结尾、位置递增的最长连续数值段长度。若 pos[x-1] < pos[x]，则 dp[x] = dp[x-1] + 1；否则 dp[x] = 1。答案 n - max(dp[x])。

样例中 pos[5]=2、pos[6]=4、pos[7]=6，因此 5,6,7 可保留，长度为 3，答案 7-3=4。直接比较输入数组相邻项 a[i-1] < a[i] 会误算“相邻位置的连续上升段”。

数值 x	1	2	3	4	5	6	7
pos[x]	5	7	1	3	2	4	6
dp[x]	1	2	1	2	1	2	3

注意 dp[x] 是按数值 x 排列的状态，不是原数组第 x 个位置的状态。

完整源代码

```
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    freopen("permutation.in", "r", stdin);
    freopen("permutation.out", "w", stdout);
    int n;
    cin >> n;
    vector<int> pos(n + 1), dp(n + 1, 1);
    for (int i = 1; i <= n; ++i) {
        int x;
        cin >> x;
        pos[x] = i;
    }
    int longest = 1;
    for (int x = 2; x <= n; ++x) {
        if (pos[x - 1] < pos[x])
            dp[x] = dp[x - 1] + 1;
        longest = max(longest, dp[x]);
    }
    cout << n - longest << '\n';
}
```

复杂度：时间 O(n)，空间 O(n)。此题要求文件输入输出；若平台改用标准输入输出，删去两行 freopen。

05 最长锯齿子序列

问题：选出最长子序列，使相邻差值正负交替。样例 1,3,5,2,4,6 的答案是 4，例如 1,5,2,6。题目保证各数字不同，因此差值不会为 0。

为什么必须分成两个状态

只知道“以 a[i] 结尾的最长长度”不够：下一次上升必须接在下降之后，下一次下降必须接在上升之后。定义 up[i] 为以 a[i] 结尾且最后一步上升的最大长度；down[i] 为以 a[i] 结尾且最后一步下降的最大长度。

枚举 j < i。若 a[j] < a[i]，新的一步上升，只能从 down[j] 接上：up[i] = max(up[i], down[j] + 1)。若 a[j] > a[i]，新的一步下降，只能从 up[j] 接上：down[i] = max(down[i], up[j] + 1)。单个数尚无方向，两种状态都用 1 作起点。

例如 1→3 已上升，不能再接 5 得到长度 3；但 1→3→2 已下降，可以接 4，形成 1→3→2→4，长度 4。

当前位置	1	2	3	4	5	6
a[i]	1	3	5	2	4	6
up[i]	1	2	2	2	4	4
down[i]	1	1	1	3	3	1

单个数同时作为两个方向的长度 1 起点；样例最大值为 4。

完整源代码：直接 DP

```
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    freopen("sub.in", "r", stdin);
    freopen("sub.out", "w", stdout);
    int n;
    cin >> n;
    vector<long long> a(n + 1);
    vector<int> up(n + 1, 1), down(n + 1, 1);
    for (int i = 1; i <= n; ++i) cin >> a[i];

    int ans = 1;
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j < i; ++j) {
            if (a[j] < a[i])
                up[i] = max(up[i], down[j] + 1);
            else
                down[i] = max(down[i], up[j] + 1);
        }
        ans = max(ans, max(up[i], down[i]));
    }
    cout << ans << '\n';
}
```

复杂度：时间 O(n²)，空间 O(n)。若 n 很大，可利用连续同向变化只需保留极值的性质，得到 O(n) 贪心写法。文件输入输出可按平台要求调整。

最后：如何判断该用哪种状态

看到的限制	优先考虑的状态	典型题
同时处理两个序列	两维前缀 dp[i][j]	LCS
只关心上一个选中元素的值	以当前位置结尾 dp[i]	LIS、最大上升和
保留元素必须数值连续	按数值 x 建状态；先求 pos[x]	排列最少移动
下一步依赖上一步的类别	按类别拆成两个状态	锯齿子序列

最容易混淆的四点

1. 子序列可以不连续，但不能改变原有顺序。排列题的“不移动部分”要求数值连续，是由最终升序且只能移到两端推出来的额外条件。

2. 严格上升统一使用 <。最大上升子序列和题可能有重复数字，相等时不能转移；排列题没有重复数字，因为它是 1 到 n 的排列。

3. LIS 的 $O(n^2)$ 写法是标准 DP；O(n log n) 写法通过保留最小结尾并二分更新。不要把 tail 直接当成输出的子序列。

4. 锯齿 DP 的“最后一步”指所选子序列最后两个数之间的变化，不要求它们在原数组中相邻。连续上升两次不能让锯齿长度连续加 1。

状态设计模板

先问四个问题：① 要最大化长度、和，还是最小化操作？② 当前状态必须记住哪个“结尾”？③ 下一次转移还依赖方向或其他类别吗？④ 答案是某个固定状态，还是所有结尾状态的最大值？